

```
midhun@localhost:~  
ab.sh      code      midhun.c  osl      Public    sh        Videos  
a.out      Desktop  mait      os.c      REC       sjf       while.sh  
awk        devi     midhun    os_lab    roundrobin sjf.c     whilew.sh  
bank       Documents mai       os_lab1   roundrobin.c sphore    XYZ  
banker.c   dog      dev       os_lab1.c salary    student.txt  
bankers    Downloads dev       os_lab.c  sample.c  table.sh  
[midhun@localhost ~]$ gcc bestfit.c -o bestfit  
[midhun@localhost ~]$ ./bestfit  
Enter number of memory blocks:  
5  
Enter size of each block:  
100 200 300 400 500  
Enter number of processes:  
4  
Enter size of each processes:  
112 113 114 115  
  
Process No.    Process Size    Block No.  
1              112            2  
2              113            3  
3              114            3  
4              115            4  
[midhun@localhost ~]$ vi bestfit.c  
[midhun@localhost ~]$
```

```
midhun@localhost:~  
ab.sh      code      midhun.c  osl      Public    sh        Videos  
a.out      Desktop  mait      os.c      REC       sjf       while.sh  
awk        devi     midhun    os_lab    roundrobin sjf.c     whilew.sh  
bank       Documents mai       os_lab1   roundrobin.c sphore    XYZ  
banker.c   dog      dev       os_lab1.c salary    student.txt  
bankers    Downloads dev       os_lab.c  sample.c  table.sh  
[midhun@localhost ~]$ gcc bestfit.c -o bestfit  
[midhun@localhost ~]$ ./bestfit  
Enter number of memory blocks:  
5  
Enter size of each block:  
100 200 300 400 500  
Enter number of processes:  
4  
Enter size of each processes:  
112 113 114 115  
  
Process No.    Process Size    Block No.  
1              112            2  
2              113            3  
3              114            3  
4              115            4  
[midhun@localhost ~]$ vi bestfit.c  
[midhun@localhost ~]$
```

```
int processSize[n], allocation[n];

// Input process sizes
printf("Enter size of each process:\n");
for (int i = 0; i < n; i++) {
    scanf("%d", &processSize[i]);
    allocation[i] = -1; // Not allocated initially
}

// First Fit Allocation
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (remainingSize[j] >= processSize[i]) {
            allocation[i] = j;
            remainingSize[j] -= processSize[i];
            break;
        }
    }
}

// Output allocation
printf("\nProcess No.\tProcess Size\tBlock No.\n");
for (int i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t\t", i + 1, processSize[i]);
    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
return 0;
```